

Habeas Check — Propuesta de Solución

Autodiagnóstico de Cumplimiento de la Ley 1581 (Fase de Diseño)

Reto CAVALTEC · Talento Tech · Documento para el equipo

1. En una frase

Construimos **Habeas Check**: una aplicación web segura y multiempresa que le permite a cualquier organización **autodiagnosticar en 5 minutos** su nivel de cumplimiento de la Ley 1581 en la fase de diseño, entregándole un **porcentaje**, un **velocímetro**, sus **brechas priorizadas** y un **plan de acción con IA** — todo en español colombiano.

Ya tenemos un **MVP funcional** (`autodiagnostico-ley1581-mvp.html`) que se abre en el navegador y demuestra el flujo completo de punta a punta.

2. Por qué esta propuesta gana el reto

El jurado evalúa **7 criterios ponderados**. La clave de nuestra estrategia es que **cada decisión de producto apunta a un criterio**. Así se reparte el puntaje y así lo atacamos:

Criterio	Peso	Cómo lo ganamos
Alineación con la Ley 1581	20%	Motor de scoring exacto (ver §4), mapeo pregunta↔principio, lenguaje normativo correcto.
Desarrollo técnico	20%	Arquitectura limpia, código tipado, motor con tests automáticos que pasan.
Seguridad	15%	OAuth + sesiones seguras + OWASP Top 10 + aislamiento multiempresa con RLS .
Uso de IA	15%	4 funciones de IA útiles ancladas en la ley: explicar, orientar, plan de acción, interpretar.
Experiencia de usuario	10%	Flujo de 3 pasos, lógica condicional, diseño institucional propio (no plantilla).
Calidad del diagnóstico	10%	Velocímetro, desglose por bloque, brechas accionables, nivel de madurez.
Innovación	10%	Simulador “¿Y si...?” , banda de madurez y exportación a PDF.

El diferenciador real: el modelo de puntuación del reto es tramposo y la mayoría de los equipos lo va a implementar mal. Nosotros lo tenemos blindado y probado. Eso solo ya nos separa en los 30% de “Alineación” + “Calidad del diagnóstico”.

3. Qué hace el MVP (lo que verán los compañeros al abrirlo)

1. **Inicio + OAuth simulado** — botones de Google y Microsoft (en el MVP están simulados; la ruta de producción está documentada).
 2. **Datos de la empresa** — nombre, NIT, sector, tamaño. Base del modelo multiempresa.
 3. **Cuestionario de 11 preguntas** en 3 bloques, con:
 - **Lógica condicional:** si no hay política (Q1=No) se bloquean Q2-Q5; si no hay oficial (Q10=No) se bloquea Q11.
 - Botón “**⚡ ¿Qué significa?**” en cada pregunta → la IA traduce el término legal a lenguaje sencillo (con respaldo offline si no hay red).
 4. **Pantalla de resultados:**
 - **Velocímetro 0-100%** con la banda de madurez (Inicial → Líder).
 - **Cumplimiento por bloque** (A/40, B/36, C/24).
 - **Brechas** ordenadas por impacto en el puntaje.
 - **Plan de acción** priorizado generado por IA.
 - **Lectura del resultado** en prosa (IA).
 - **Simulador “¿Y si...?”:** active una mejora y vea en vivo cuánto sube el puntaje.
 - **Exportar a PDF** (imprimir).
-

4. El corazón: el modelo de puntuación (no improvisar)

El reto define un modelo con **tres reglas no obvias**. Implementarlas exactamente es lo que más puntos da y donde más equipos fallan.

Bloques y pesos (total 100%):

- **Bloque A — Política de datos (máx. 40%):** Q2, Q3, Q4, Q5 = 10% cada una.
- **Bloque B — Privacidad desde el diseño (máx. 36%):** Q6, Q7, Q8 = 12% cada una.
- **Bloque C — Gobernanza (máx. 24%):** Q9 = 16%, Q10 = 8%.

Las tres reglas:

- 1. **Q1 es pregunta “padre” y NO tiene peso propio:** hereda la suma de sus hijas Q2-Q5. → El bloque A es simplemente la suma de Q2-Q5. (*Error común: sumar 40% extra por responder Q1 “Sí”.*)
- 2. **Q11 es complementaria y NUNCA suma:** solo matiza a Q10 (“tiene oficial, pero sin designación formal”). (*Error común: dejar que Q11 sume o reste.*)
- 3. **Total = suma directa de los pesos ganados en Q2-Q10,** con tope de 100%. Q1 y Q11 quedan fuera del cálculo.

Vectores de prueba que nuestro motor pasa (esto es lo que mostramos como evidencia técnica):

Caso	Esperado
Todo “No”	0%
Todo “Sí” (Q2-Q10)	100%
Solo Q2 y Q3	20%
Q1=Sí pero hijas “No”	0% (el padre no hereda nada)
Todo “Sí” excepto Q11=No	100% (la complementaria no resta)
Q10=No, Q11=Sí (estado ilegal)	Se ignora Q11

Bandas de madurez (diferenciador de innovación): 0-24 Inicial · 25-49 Básico · 50-74 Gestionado · 75-94 Optimizado · 95-100 Líder.

5. Arquitectura (del MVP a producción)

MVP actual: un archivo HTML autocontenido (HTML + CSS + JS), sin dependencias, abre con doble clic. Perfecto para demostrar el flujo y la lógica sin montar infraestructura.

Producción propuesta (Nivel 3):

- **Frontend:** Next.js + TypeScript + Tailwind. Despliegue en **Vercel**.
- **Auth:** OAuth real (Google + Microsoft) vía Supabase Auth, sesiones seguras.
- **Base de datos:** Supabase Postgres con **Row-Level Security** → cada empresa solo ve sus evaluaciones.
- **IA:** Anthropic API desde una **ruta de servidor** (la llave nunca llega al navegador).
- **Reportes:** PDF server-side + histórico de evaluaciones por empresa.

Modelo de datos (resumen): `companies` (empresa) → muchas `evaluations` → muchas `answers`; `memberships` define el **rol** del usuario (administrador / evaluador / auditor) sobre cada empresa; `recommendations` cachea las salidas de IA para que los informes sean reproducibles.

6. Seguridad (15% del puntaje, y es una app *de* protección de datos)

- **OAuth** con Authorization Code + PKCE; nada de tokens de larga vida en el navegador.
 - **Roles** verificados en el servidor (el auditor es solo lectura, no basta con ocultar botones).
 - **RLS multiempresa:** el aislamiento se hace en la base de datos, no solo en el código. Una fuga entre empresas en una herramienta de privacidad sería descalificante.
 - **OWASP Top 10:** control de acceso, validación de entradas (NIT, respuestas), consultas parametrizadas, secretos solo en el servidor, cabeceras de seguridad.
 - **Privacy by design en nuestra propia app:** recogemos solo lo mínimo de la empresa (nombre, NIT, sector, tamaño).
-

7. La IA, con criterio (no un chatbot pegado)

Cuatro funciones, cada una con un prompt enfocado y **respaldo determinista** si la API falla (la demo nunca se queda en blanco):

1. **Explicar** — traduce la pregunta legal a lenguaje sencillo.
2. **Orientar** — qué se necesita realmente para responder “Sí” con honestidad (esto sube la calidad del diagnóstico).
3. **Plan de acción** — acciones priorizadas para cerrar brechas, de mayor a menor impacto, en JSON estructurado.
4. **Interpretar** — lectura en prosa del resultado y las 2 prioridades clave.

Regla de oro: **el número siempre lo calcula el motor, no la IA**. La IA explica el número; no lo inventa.

8. Plan de trabajo sugerido para el equipo

Frente	Responsable (a definir)	Entregable
Motor de scoring + tests	—	Ya hecho en el MVP; portar a TypeScript.
Frontend / UX	—	Migrar el MVP a Next.js, pulir responsive y accesibilidad.
Auth + multiempresa + RLS	—	Supabase: login OAuth, tablas, políticas RLS, roles.
Integración IA (servidor)	—	Ruta <code>/api/ai</code> , los 4 prompts, caché en BD.
Reportes + histórico	—	PDF y línea de tiempo de evaluaciones por empresa.
Despliegue + demo	—	Vercel + datos de prueba + guion de presentación.

Sugerencia de alcance por tiempo: si llegamos justos, construimos la **arquitectura de Nivel 3** pero entregamos el **set de funciones de Nivel 2** con costuras limpias para subir. Siempre tener una demo que funcione en cada checkpoint.

9. Qué mostramos en la presentación (guion de 4 minutos)

1. **Abrir el MVP** y hacer el flujo en vivo: empresa → responder con lógica condicional → resultados.
2. Mostrar el **velocímetro** moverse y el **simulador “¿Y si...?”** (activar una mejora y ver subir el puntaje).
3. Tocar “¿Qué significa?” en una pregunta para enseñar la IA.
4. Exportar a **PDF** en vivo.
5. Cerrar con la lámina de **scoring** (§4): “este modelo es la trampa del reto; nosotros lo tenemos probado”.

Archivos del equipo: `autodiagnostico-Ley1581-mvp.html` (MVP demostrable) y este documento de propuesta.

